

02장 컴퓨터의 구조와 성능 향상

학습목표

- 폰노이만 구조의 특징을 이해한다.
- CPU와 메모리의 구성 및 동작 방식을 살펴본다.
- 컴퓨터 성능을 향상시키는 다양한 기술을 학습한다.

목차

1. 컴퓨터의 기본 구성
2. CPU와 메모리
3. 컴퓨터 성능 향상 기술

01

컴퓨터의 기본 구성

01 컴퓨터의 기본 구성

1. 하드웨어의 구성

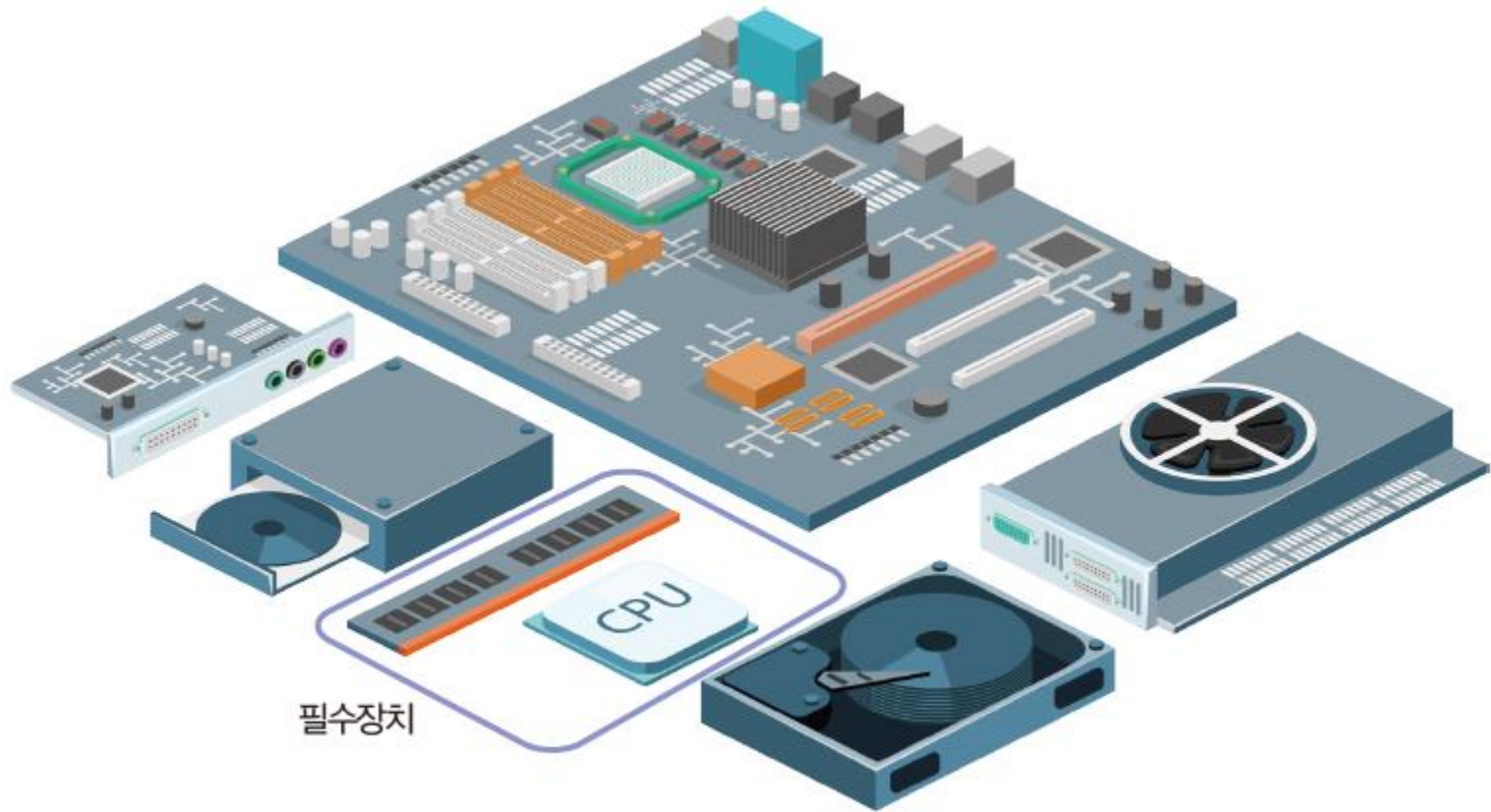


그림 2-1 컴퓨터 하드웨어

01 컴퓨터의 기본 구성

1. 하드웨어의 구성

◆ 컴퓨터의 필수장치

- 중앙처리장치(CPU), 메인 메모리

◆ 컴퓨터의 주변장치

- 입력장치, 출력장치, 저장장치



그림 2-2 저장장치의 종류

01 컴퓨터의 기본 구성

1. 하드웨어의 구성

◆메인보드와 버스

- 메인보드: 장치들을 연결하고 전원을 공급하는 회로 기판
 - 카드 설치 가능
- 버스: 장치 간 데이터 통로

01 컴퓨터의 기본 구성

2. 폰노이만 구조

◆ 폰노이만 구조의 특징

- CPU, 메모리, 주변장치가 버스로 연결됨

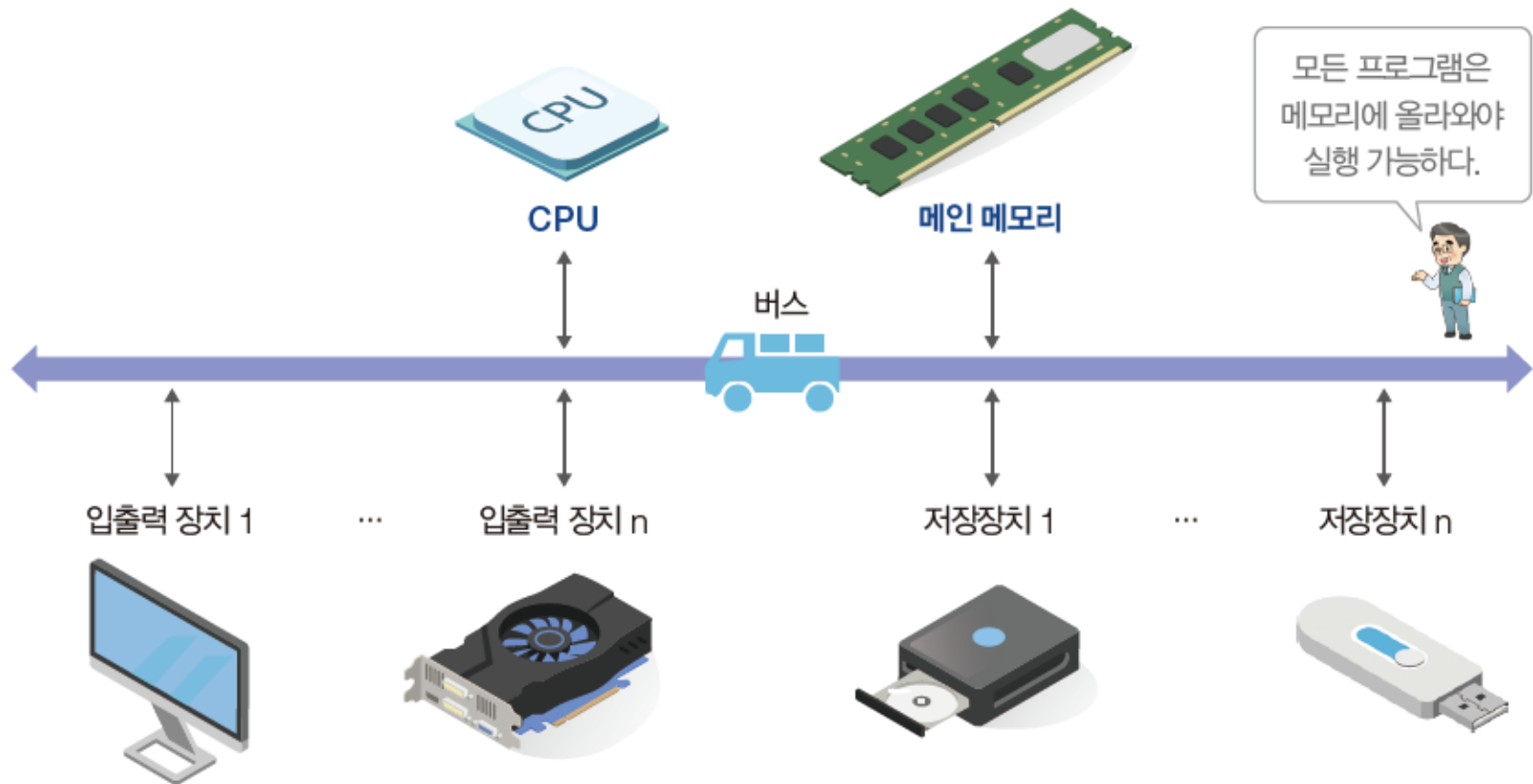


그림 2-3 폰노이만 구조

01 컴퓨터의 기본 구성

2. 폰노이만 구조

◆ 폰노이만 구조의 특징

- CPU와 메모리가 협업하여 작업을 수행
- 메모리가 유일한 작업 공간
- 모든 프로그램은 메모리에 올라와야만 실행 가능

01 컴퓨터의 기본 구성

2. 폰노이만 구조

◆ 요리사 모형으로 폰노이만 구조 이해하기

- 요리사(CPU)가 조리하려면 창고(저장장치)의 재료를 조리대(메모리)로 가져와야 함
- 요리사는 오직 조리대(메모리)에서만 조리
- 요리사는 조리 방법을 결정(프로세스 관리)하고,
조리대 위의 재료를 관리(메모리 관리)하며, 보관 창고를 정리(저장장치 관리)



그림 2-4 요리사 모형

01 컴퓨터의 기본 구성

2. 폰노이만 구조

- 메모리 크기가 작업 속도에 영향을 미침
- 작업 공간이 작으면 재료를 조리대까지 자주 가져오고 정리하느라 느려짐

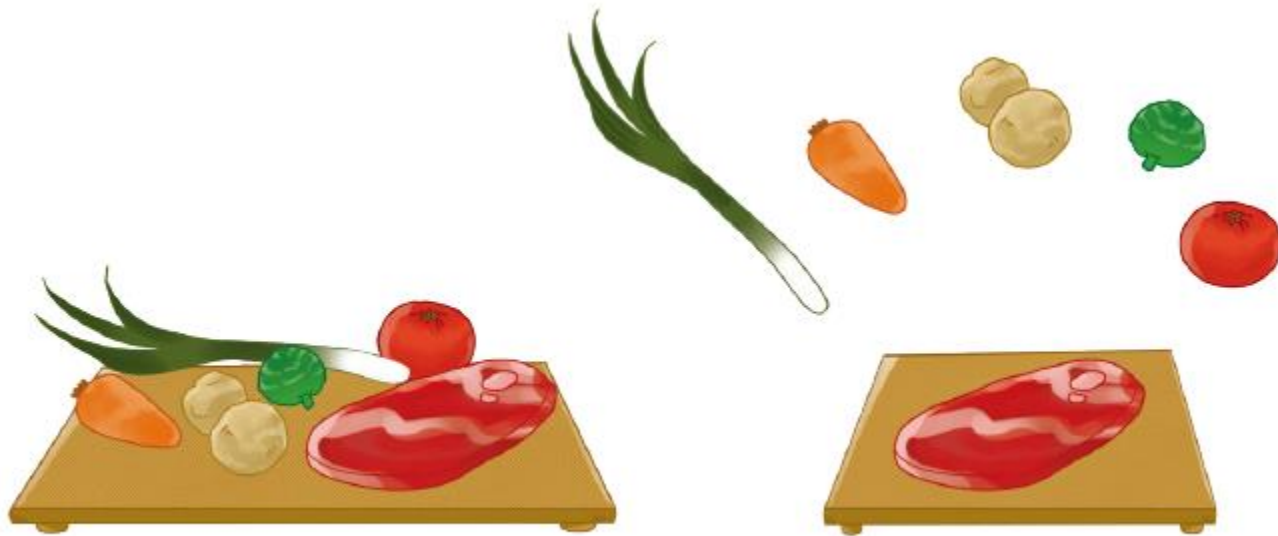


그림 2-5 조리대의 크기와 작업 속도

01 컴퓨터의 기본 구성

3. 기초 용어 정리

◆ 데이터 크기의 단위

- 비트: 데이터의 기본 단위, 8비트=1바이트

표 2-1 데이터 크기의 단위

단위	표기	바이트 대비 크기(2진)	바이트 대비 크기(10진)
Byte(바이트)	B	1B	1B
Kilobyte(킬로바이트)	KB	$2^{10}B$	10^3B
Megabyte(메가바이트)	MB	$2^{20}B$	10^6B
Gigabyte(기가바이트)	GB	$2^{30}B$	10^9B
Terabyte(테라바이트)	TB	$2^{40}B$	$10^{12}B$
Petabyte(페타바이트)	PB	$2^{50}B$	$10^{15}B$

01 컴퓨터의 기본 구성

3. 기초 용어 정리

◆ 클럭과 헤르츠

- 클럭이 틱(또는 펄스)이라는 신호를 만들고, 컴퓨터의 모든 작업은 신호에 맞춰 진행
- 헤르츠: 1초 동안 발생한 펄스의 횟수를 나타내는 단위

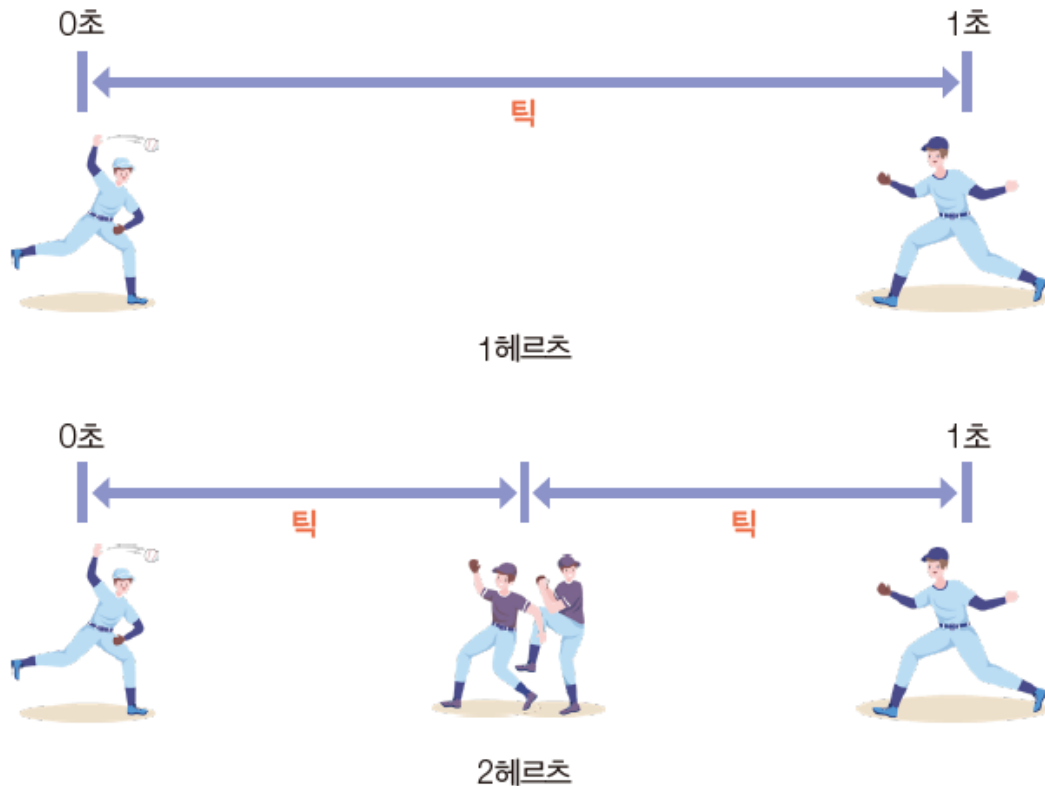


그림 2-6 헤르츠(Hz)의 의미

01 컴퓨터의 기본 구성

3. 기초 용어 정리

◆ 시스템 버스와 CPU 버스

- 시스템 버스: 메모리와 주변장치를 연결
- CPU 버스: CPU 내부에서 구성 요소를 연결

표 2-2 하드웨어 사양 예시

부품	사양
CPU	인텔 코어 i7(6코어, 12스레드, 기본 3.7GHz, 최대 5.0GHz, 캐시 20MB)
메인보드	FSB 3,200MHz
메모리	DDR4 SDRAM 4GB(3,200MHz)
그래픽카드	RTX 3060 12GB, 베이스클럭 1,530MHz
하드디스크(HDD)	1TB, 7200rpm, 256MB

01 컴퓨터의 기본 구성

3. 기초 용어 정리

◆ 프로그래밍 언어

- 기계어, 저급 언어와 고급 언어
- 컴퓨터에 명령을 내리려면 고급 언어로 소스 코드를 기계어로 번역해야 함

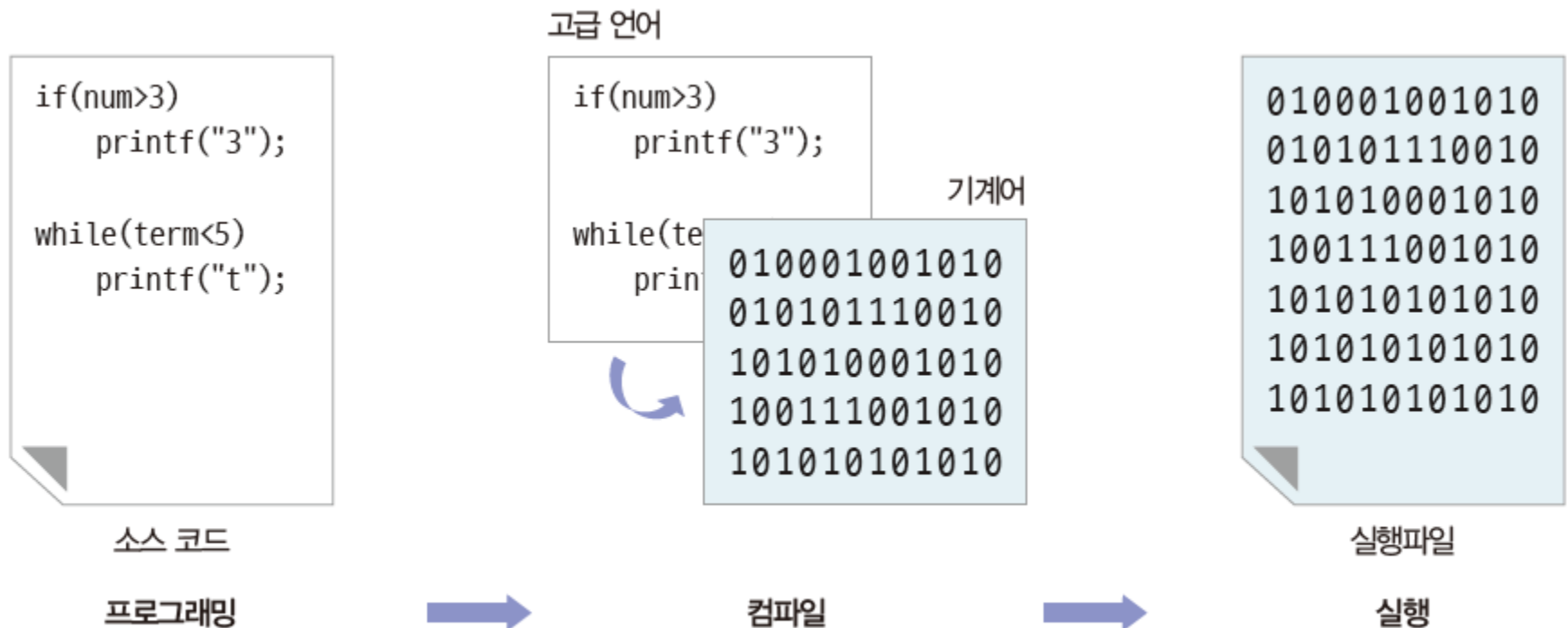


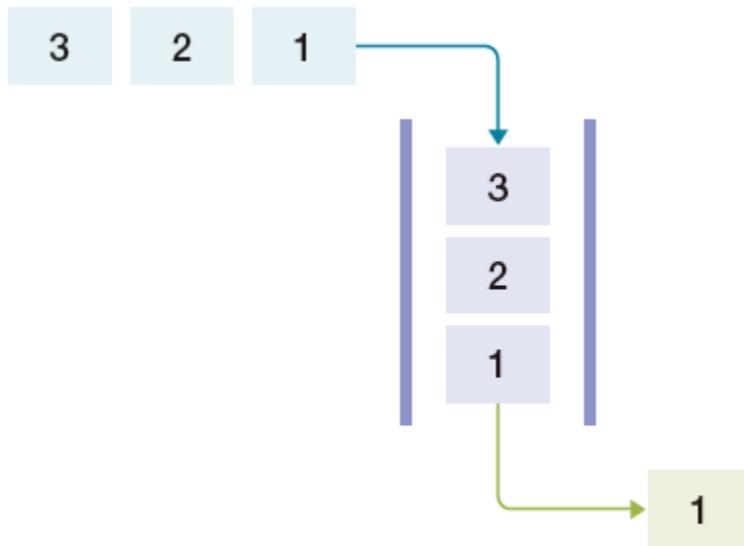
그림 2-7 컴파일 방식

01 컴퓨터의 기본 구성

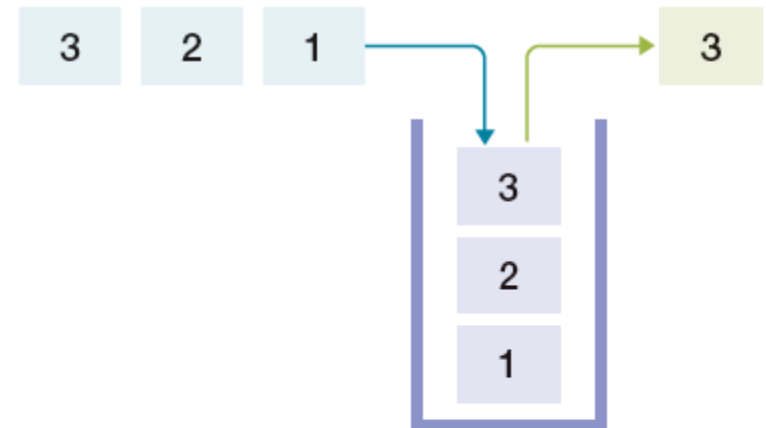
3. 기초 용어 정리

◆ 자료구조

- 큐, 스택
- 배열
- 연결 리스트



(a) 큐



(b) 스택

그림 2-8 큐와 스택

02

CPU와 메모리

02 CPU와 메모리

1. CPU의 구성과 동작 방식

◆ CPU의 구성 장치

- 산술논리 연산장치
- 제어장치
- 레지스터



(a) 산술논리 연산장치



(b) 제어장치



(c) 레지스터

그림 2-9 CPU의 구성 장치

02 CPU와 메모리

1. CPU의 구성과 동작 방식

◆ CPU의 명령어 처리 과정

코드 2-1 덧셈 연산의 유사코드

```
01  int D2 = 2;
02  int D3 = 3;
03  int sum;
04
05  sum = D2 + D3;
```

코드 2-2 덧셈 연산의 어셈블리어 유사코드

```
01  LOAD mem(100), R2      // 메모리 100번지 데이터를 R2로 이동
02  LOAD mem(120), R3      // 메모리 120번지 데이터를 R3로 이동
03  ADD R5, R2, R3         // R2와 R3를 더한 후 R5에 임시 저장
04  MOVE R5, mem(160)      // R5 결과를 160번지에 저장
```

02 CPU와 메모리

1. CPU의 구성과 동작 방식

◆ CPU의 명령어 처리 과정

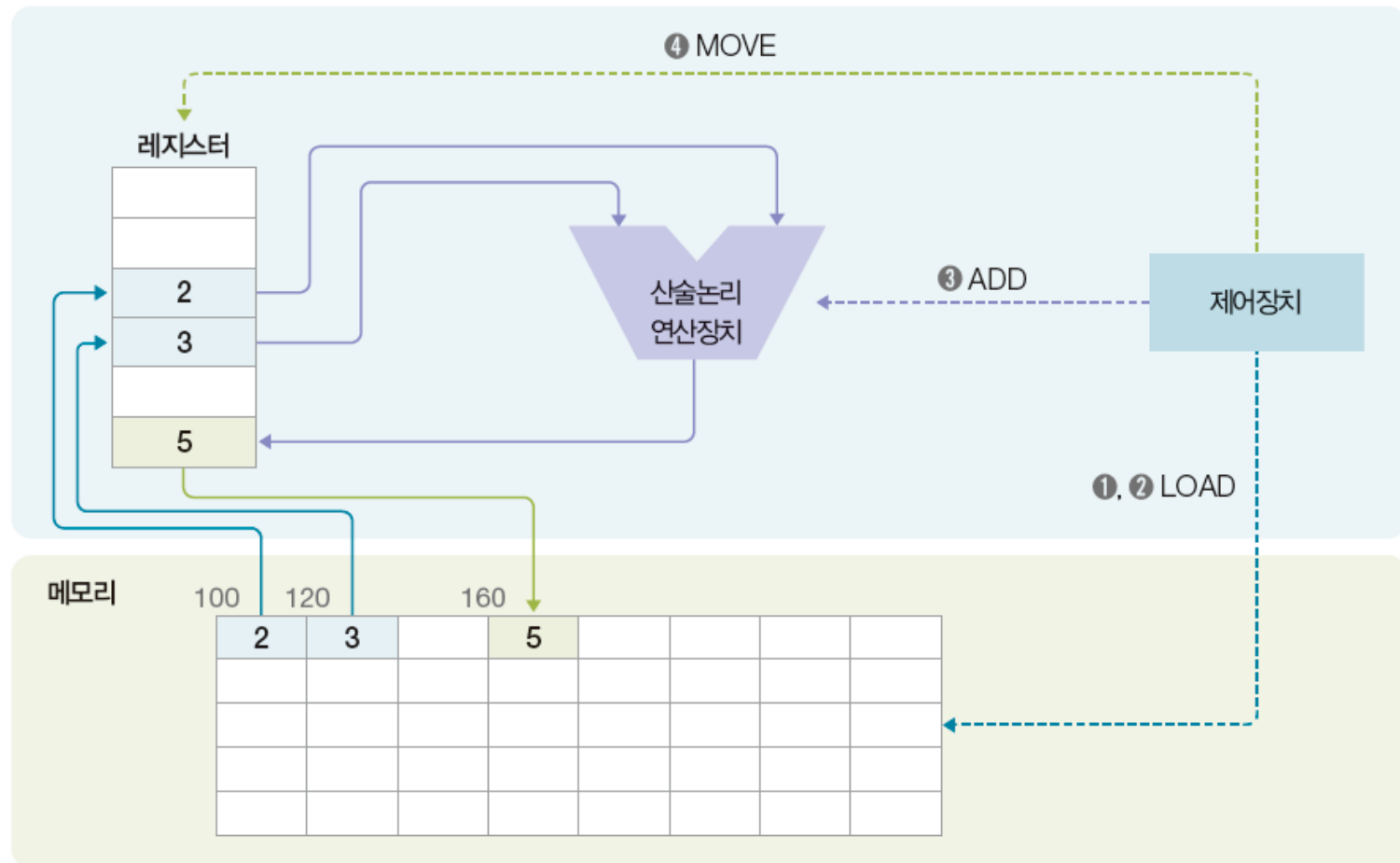


그림 2-10 CPU의 명령어 처리 과정

02 CPU와 메모리

1. CPU의 구성과 동작 방식

◆CPU의 명령어 처리 과정

CPU의 명령어 처리 과정	코드 2-2	설명
메모리에서 값 가져오기	1행, 2행	① 제어장치가 메모리의 100번지에 있는 값(2)을 레지스터 R2로 옮기라는 명령을 내린다. ② 제어장치가 메모리의 120번지에 있는 값(3)을 레지스터 R3로 옮기라는 명령을 내린다.
연산 수행	3행	③ 산술논리 연산장치가 제어장치의 명령에 따라 레지스터 R2와 R3에 저장된 값을 더한다. 연산 결과인 5는 레지스터 R5에 임시 저장한다.
결과 저장	4행	④ 제어장치가 R5에 저장된 값(5)을 메모리의 160번지로 옮기라는 명령을 내린다.

2. 레지스터 자세히 알기

◆ 레지스터의 종류

- 사용자 가시 레지스터
 - 사용자 프로그램에 의해 변경됨
 - 데이터 레지스터
 - 주소 레지스터
- 사용자 불가시 레지스터: 특수 레지스터
 - 특정 용도로 사용되며, 사용자가 내용을 임의로 변경할 수 없음
 - 프로그램 카운터
 - 명령어 레지스터
 - 메모리 주소 레지스터
 - 메모리 버퍼 레지스터
 - 프로그램 상태 레지스터

02 CPU와 메모리

◆ 레지스터의 동작 과정

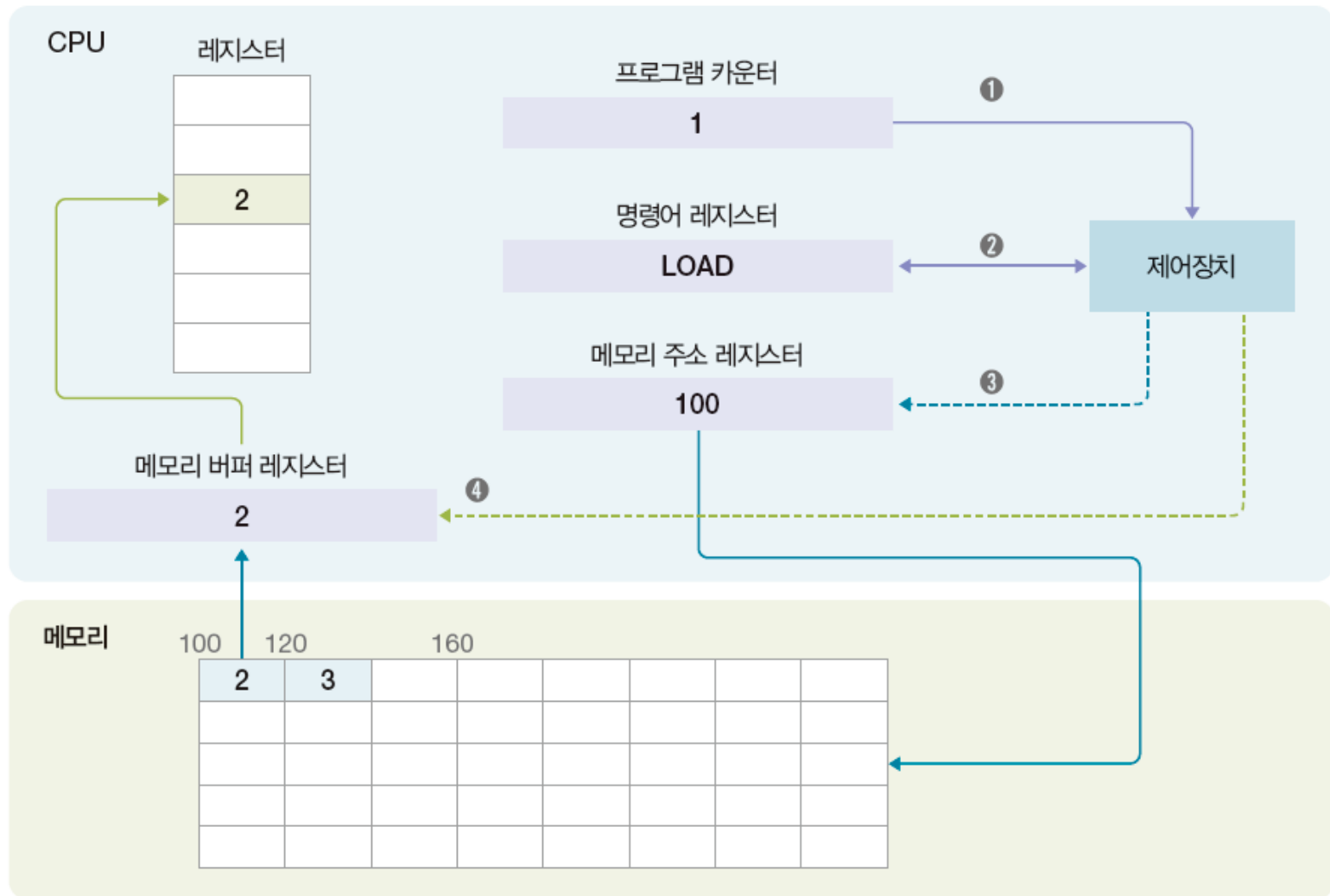


그림 2-11 `LOAD mem(100), R2`의 실행 과정

02 CPU와 메모리

2. 레지스터 자세히 알기

◆ 레지스터의 동작 과정

- D2-D3이 양수인지 음수인지 PSR에 저장됨
- 제어장치가 결과를 받아 몇 행으로 이동할지 결정

코드 2-3 분기 조건이 있는 유사코드

```
01  if (D2 - D3 > 0)
02      goto 100;          # D2 - D3의 결과가 양수이면 100행으로 이동
03  else
04      goto 200;          # D2 - D3의 결과가 음수이면 200행으로 이동
```

02 CPU와 메모리

3. 버스의 구조

◆ 버스의 종류와 특징

- 시스템 버스는 CPU와 메모리, 주변장치 간 데이터를 주고받는 통로

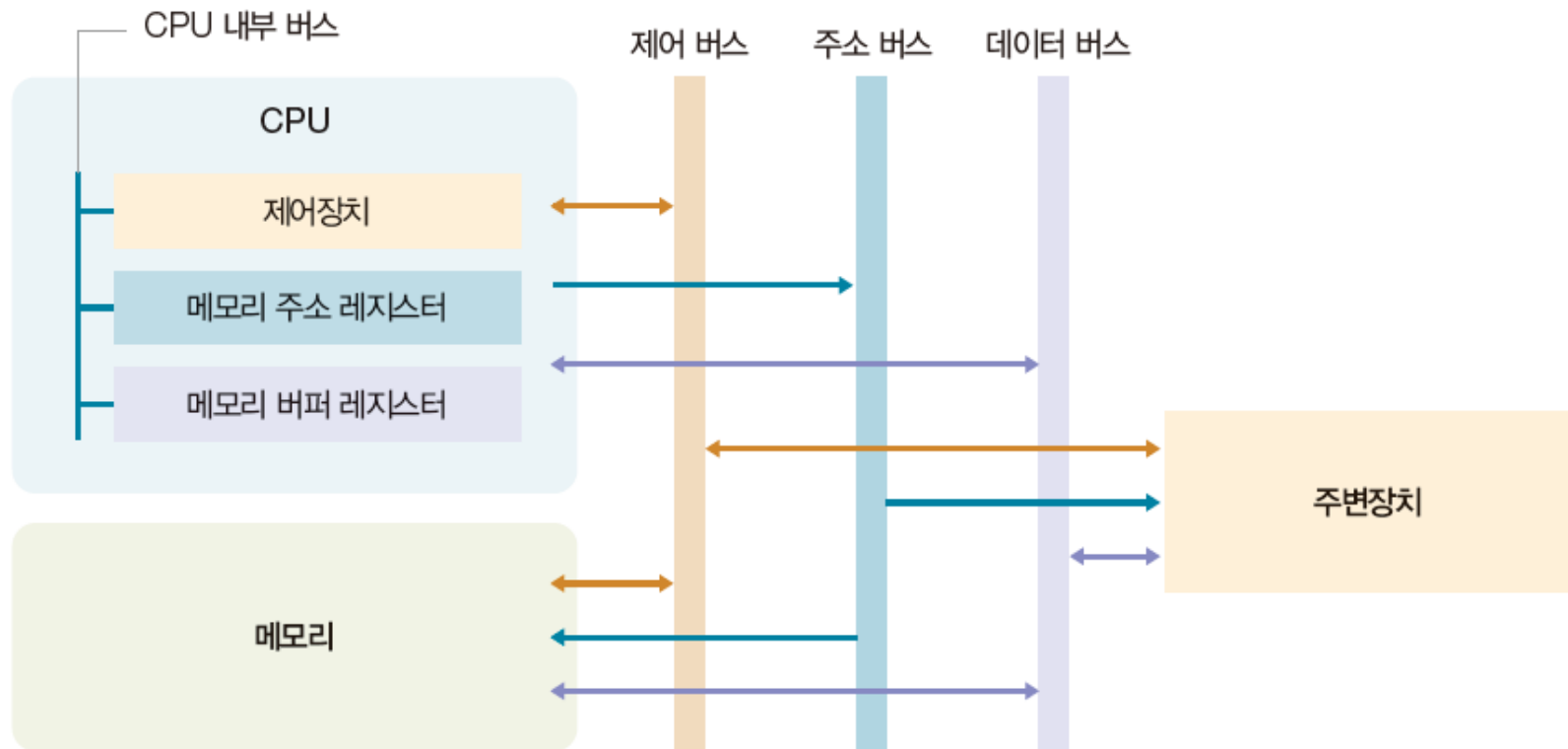


그림 2-12 버스의 종류와 구조

02 CPU와 메모리

3. 버스의 구조

◆ CPU의 비트 수와 대역폭

- CPU 비트 수: CPU가 한 번에 처리할 수 있는 데이터 크기
 - 워드
- 대역폭: 버스에서 데이터를 한 번에 전달할 수 있는 최대 크기

02 CPU와 메모리

4. 메모리의 종류

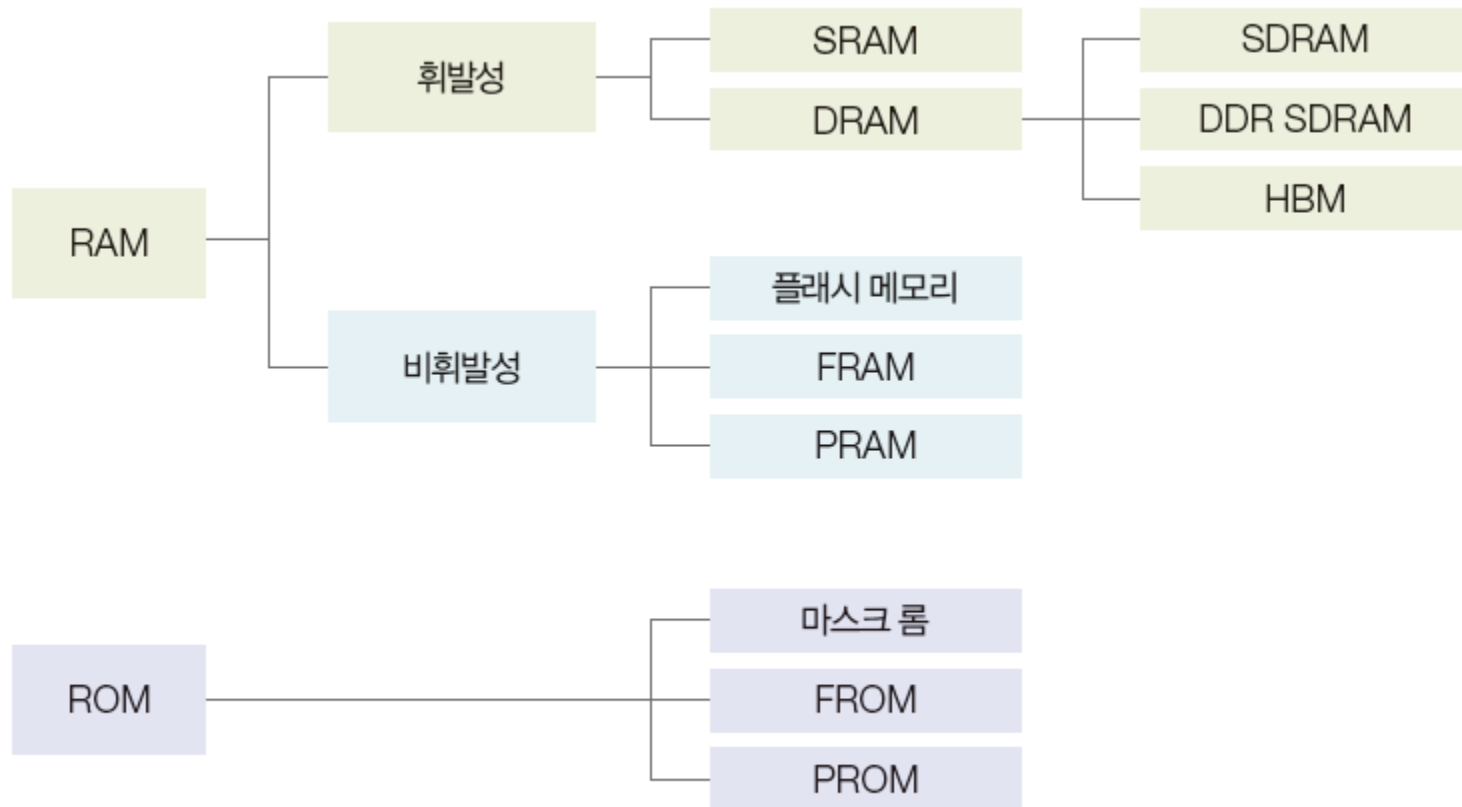


그림 2-13 메모리의 종류

02 CPU와 메모리

4. 메모리의 종류

◆ 램: 휘발성 메모리

- DRAM과 SRAM
- SDRAM과 DDR SDRAM
- HBM

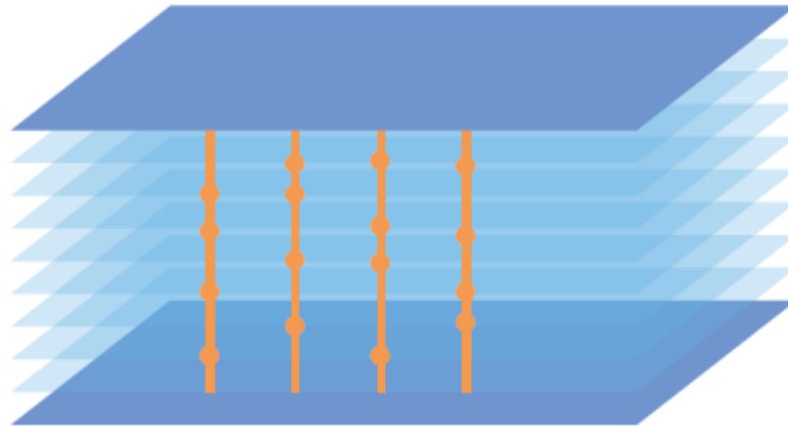


그림 2-14 초광대역 메모리

02 CPU와 메모리

4. 메모리의 종류

◆ 램: 비휘발성 메모리

- 플래시 메모리, FRAM, PRAM

◆ 롬

- 마스크 롬, PROM, EPROM
 - 바이오스가 저장되는 곳

02 CPU와 메모리

4. 메모리의 종류

◆ 저장장치 계층구조

- 빠른 동작 속도와 큰 용량을 동시에 확보하는 수단
 - 메모리 동작 속도: 레지스터 > 캐시 메모리 > 메인 메모리 > 보조 저장장치
 - 가격 대비 용량: 보조 저장장치 > 메인 메모리 > 캐시메모리 > 레지스터

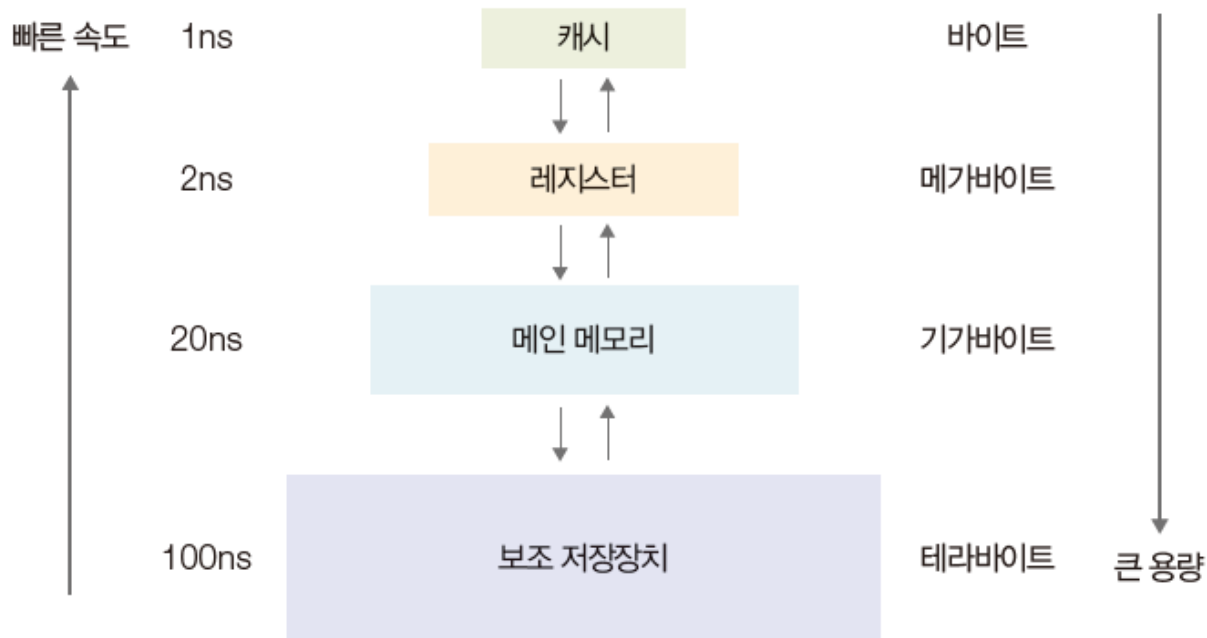


그림 2-15 저장장치 계층구조

02 CPU와 메모리

5. 폰노이만 구조와 부팅 과정

- 부팅: 운영체제를 메모리에 올리는 과정

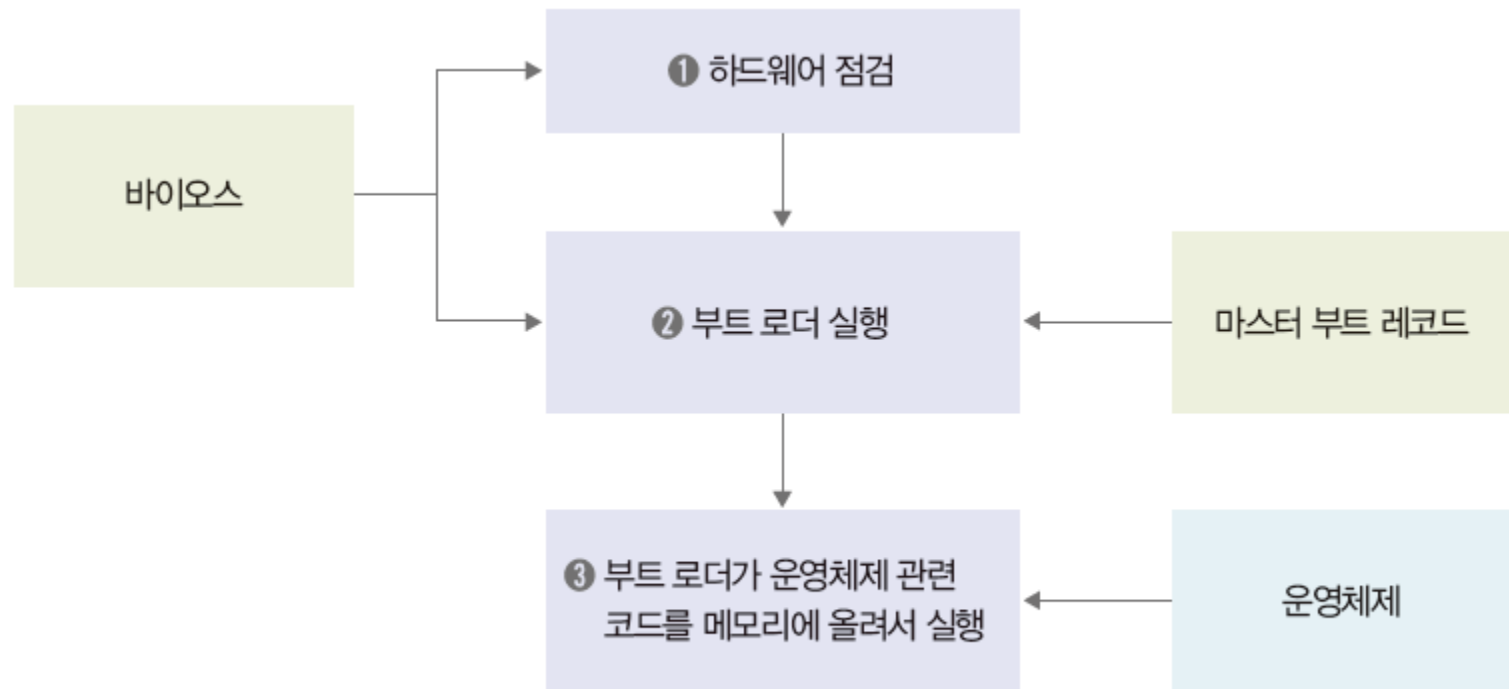


그림 2-16 부팅 과정

02 CPU와 메모리

5. 폰노이만 구조와 부팅 과정



(a) 윈도우 운영체제를 메모리에 탑재 중



(b) 유닉스 운영체제를 메모리에 탑재 중

그림 2-17 운영체제별 부팅 중 화면

03

컴퓨터 성능 향상 기술

03 컴퓨터 성능 향상 기술

1. 버퍼

- 속도가 다른 두 장치 사이에서 데이터를 임시로 저장하여 속도 차이를 줄이는 하드웨어 또는 소프트웨어

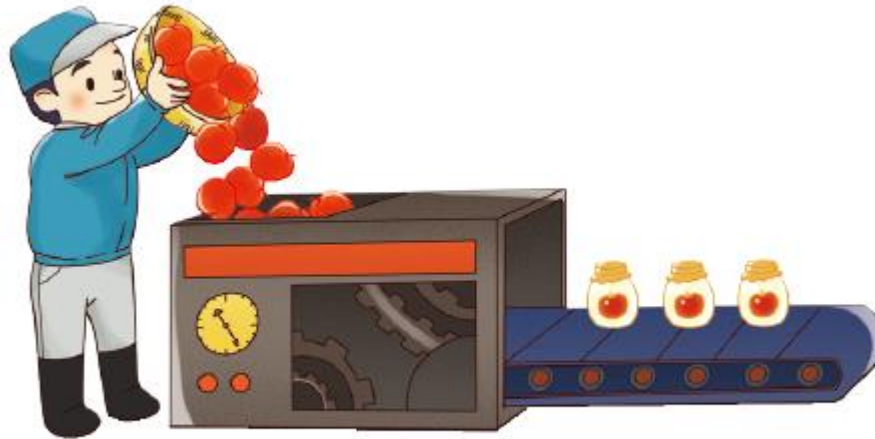


그림 2-18 버퍼의 개념

03 컴퓨터 성능 향상 기술

1. 버퍼

◆스풀

- 프린터 출력에 사용되는 스푼러에 인쇄 내용이 저장되어 있어 다른 작업 수행 가능

◆캐시

- CPU에 내장되어 CPU와 메모리의 속도 차이를 완화하는 초고속 메모리
 - 앞으로 사용 예상되는 데이터를 미리 가져다놓음(prefetch)



그림 2-19 캐시의 개념

03 컴퓨터 성능 향상 기술

1. 버퍼

◆ 캐시

- 캐시 히트: 메모리에 접근하지 않아도 돼서 작업 속도가 빠름
- 캐시 미스: 메모리에서 데이터를 가져오느라 작업 속도가 느려짐

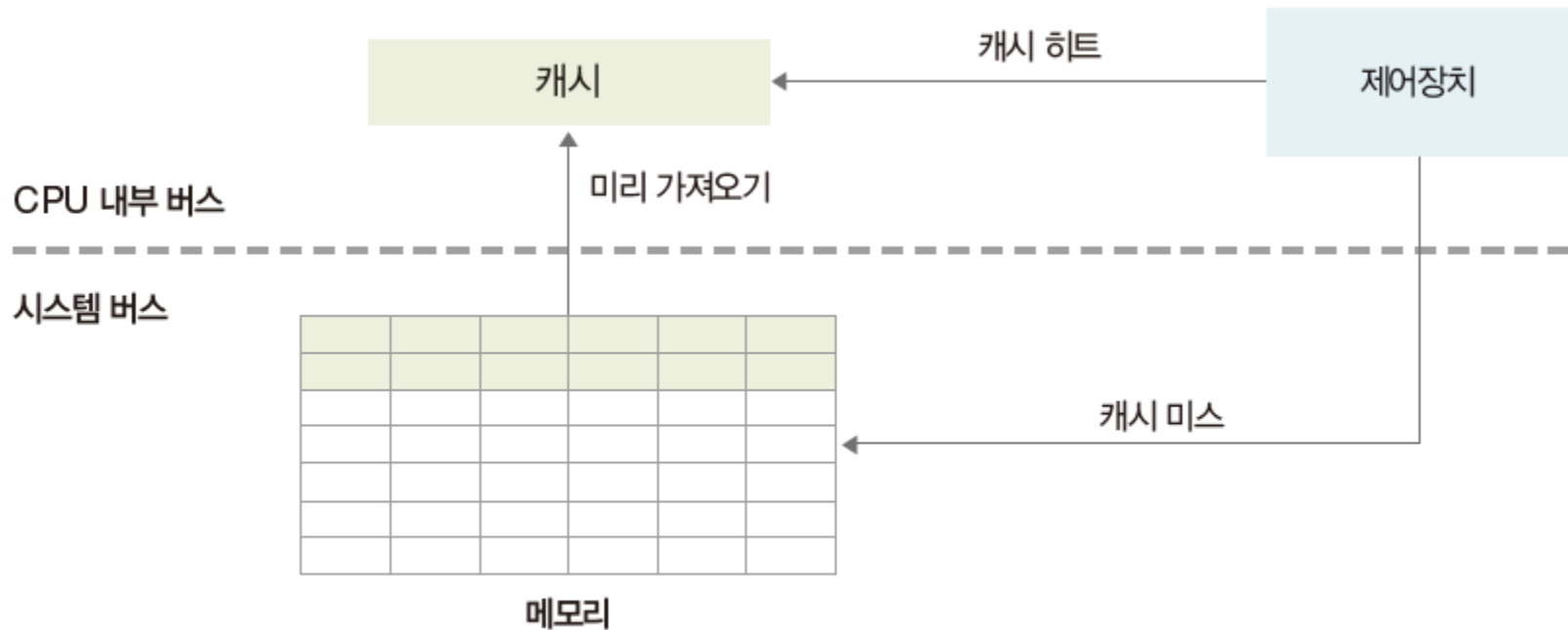


그림 2-20 캐시에 데이터 미리 가져오기

03 컴퓨터 성능 향상 기술

1. 버퍼

◆ 캐시 히트율을 높이는 방법

- 캐시 크기 증대
- 지역성 원리 이용

◆ 캐시의 종류

- 일반 캐시:
모든 자료를 미리 가져옴
- 특수 캐시:
명령어와 데이터를 구분

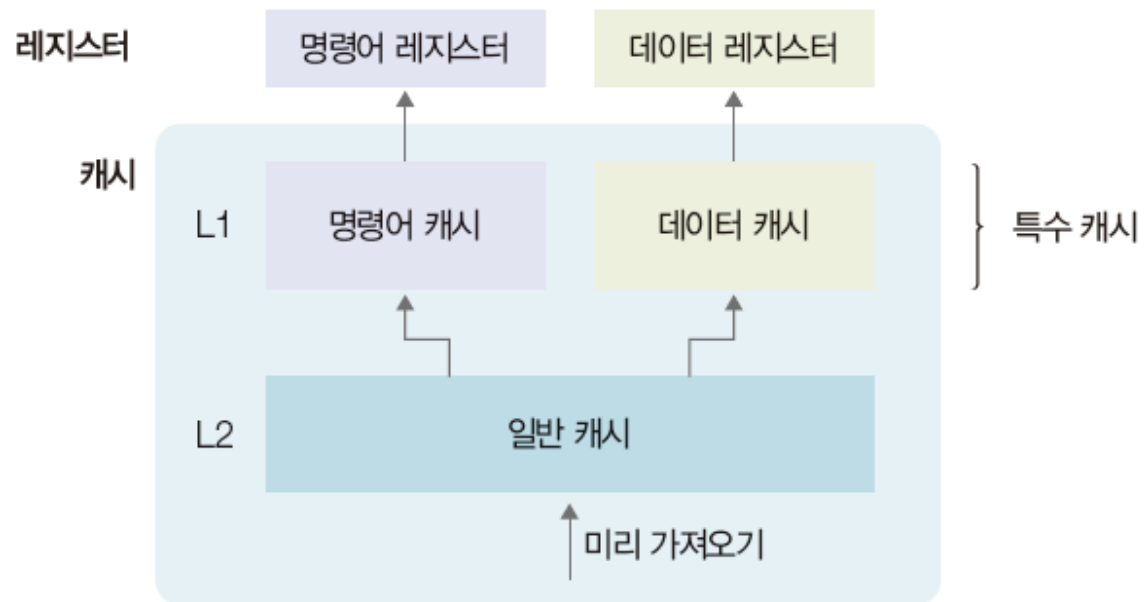


그림 2-21 L1 캐시와 L2 캐시

03 컴퓨터 성능 향상 기술

2. 인터럽트와 DMA

◆ 입출력 관리자와 인터럽트

- 폴링: CPU가 직접 입출력 장치에서 데이터 가져오거나 내보냄
- 인터럽트: CPU는 자신의 작업을 수행,
입출력이 완료되면 입출력 관리자가 인터럽트 신호를 보냄



(a) 폴링 방식

(b) 인터럽트 방식

그림 2-22 인터럽트 방식의 입출력

03 컴퓨터 성능 향상 기술

2. 인터럽트와 DMA

◆ 인터럽트 방식의 입출력 과정

- 인터럽트 번호: 여러 입출력 장치를 구분하는 고유 번호
- 인터럽트 벡터: 여러 개의 인터럽트를 동시에 CPU에 전달하려 배열로 묶은 것
- 인터럽트 발생 시 커널은 작업을 중단하고 인터럽트를 처리

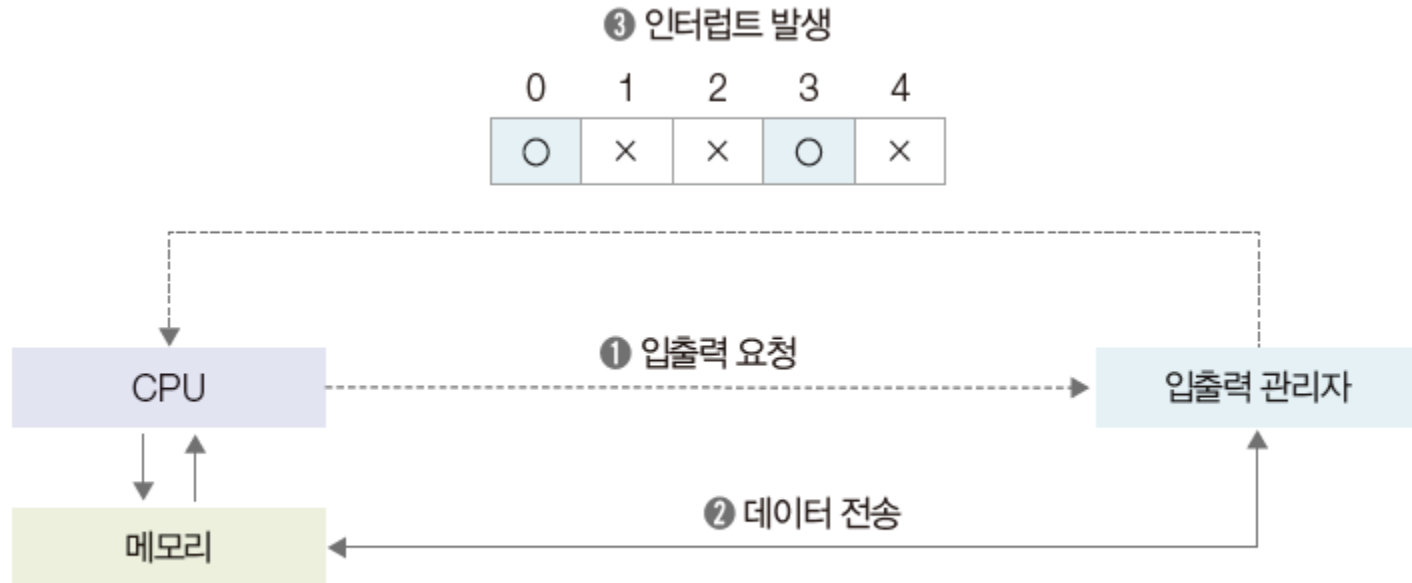


그림 2-23 인터럽트 방식의 입출력 과정

03 컴퓨터 성능 향상 기술

2. 인터럽트와 DMA

◆ 직접 메모리 접근과 메모리 맵 입출력

- 직접 메모리 접근: CPU의 개입 없이도 입출력 장치가 메모리에 접근할 권한이 있음
- 메모리 맵 입출력: CPU와 입출력 관리자가 사용하는 메모리 공간을 분리

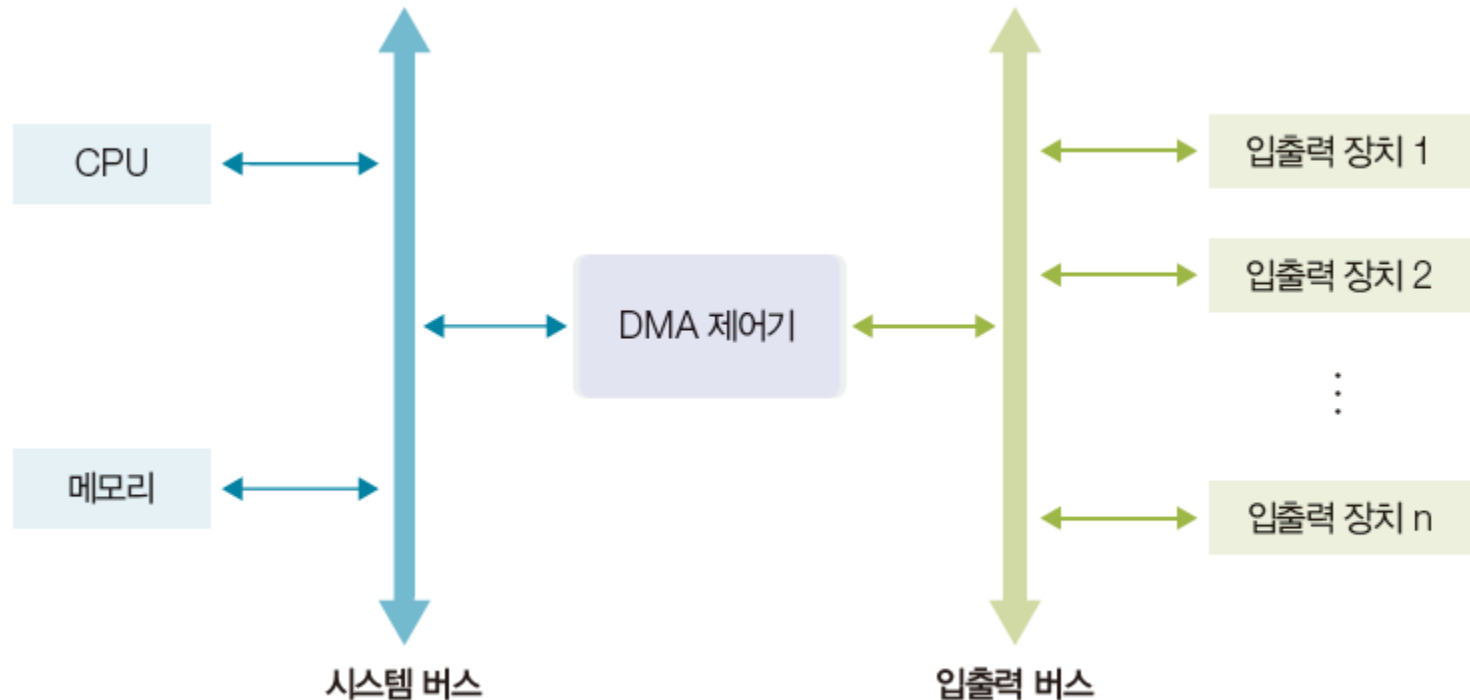


그림 2-24 직접 메모리 접근 방식 입출력

03 컴퓨터 성능 향상 기술

2. 인터럽트와 DMA

◆ 사이클 훔치기

- CPU와 입출력 장치가 동시에 메모리에 접근하려 할 때 입출력 장치가 먼저 사용

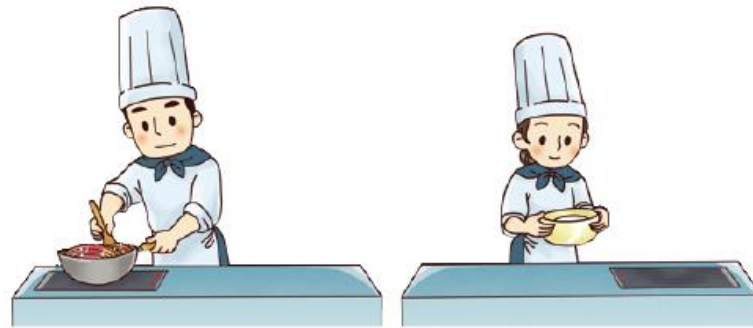
03 컴퓨터 성능 향상 기술

3. 멀티프로세싱

- 컴퓨터가 한 번에 여러 작업을 처리하게 하여 성능을 높임



(a) 단일 프로세서



(b) 멀티프로세서



(c) 멀티코어



(d) 명령어 병렬처리

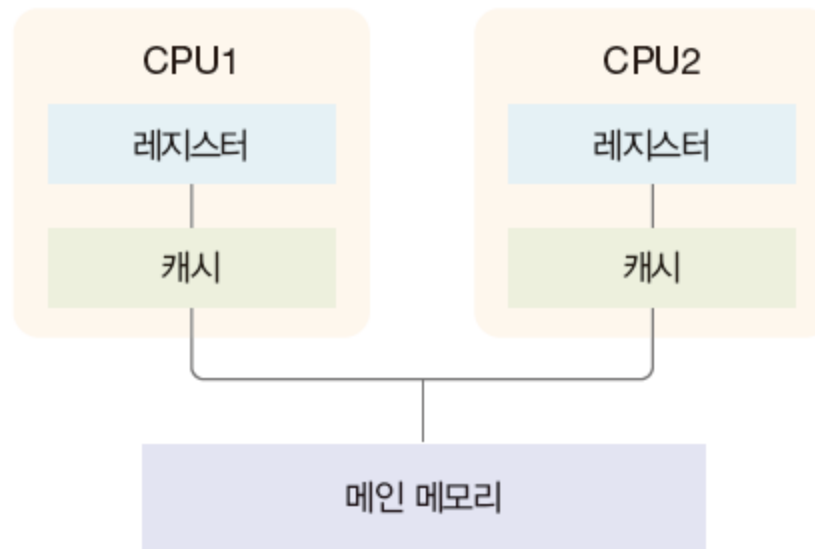
그림 2-25 단일 프로세서 시스템과 세 가지 멀티프로세싱 기술 비교

03 컴퓨터 성능 향상 기술

3. 멀티프로세싱

◆ 멀티프로세서 시스템

- 2개 이상의 프로세서 사용
 - 각 CPU에 독립된 레지스터와 캐시 있음
 - 모든 CPU가 메인 메모리를 공유
- 병목현상



멀티프로세서 시스템은
여러개의 CPU로 구성된다.



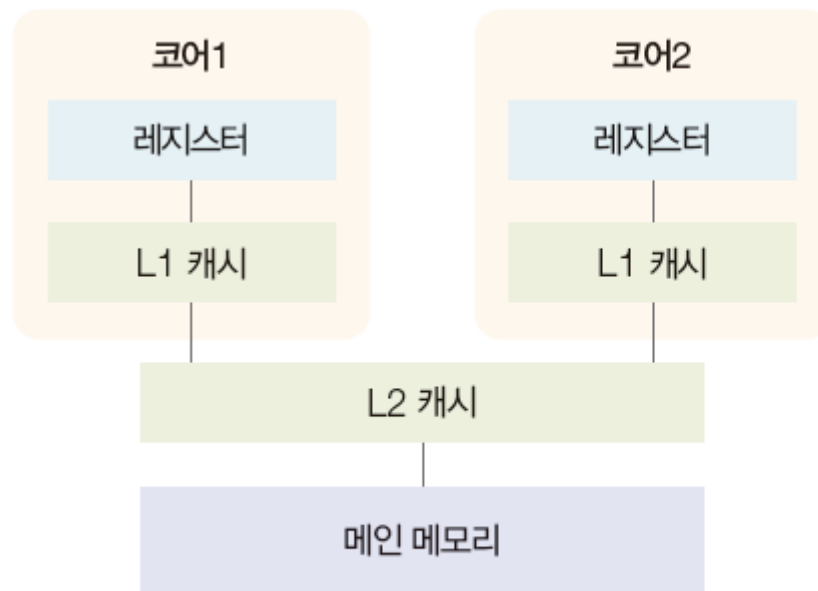
그림 2-26 멀티프로세서 시스템

03 컴퓨터 성능 향상 기술

3. 멀티프로세싱

◆ 멀티코어 시스템

- 한 개의 프로세서 안에 여러 개의 코어를 구성
- 보드 설계를 변경할 필요가 없음



멀티코어 시스템에는
하나의 프로세서 안에
여러 개의 코어가 있다.



그림 2-27 멀티코어 시스템

03 컴퓨터 성능 향상 기술

3. 멀티프로세싱

◆ 멀티코어 시스템

- 기본적으로 각 코어가 레지스터와 L1 캐시를 가짐
 - 여러 개의 코어가 L2 캐시를 공유, 모든 코어가 L3 캐시를 공유하는 계층구조

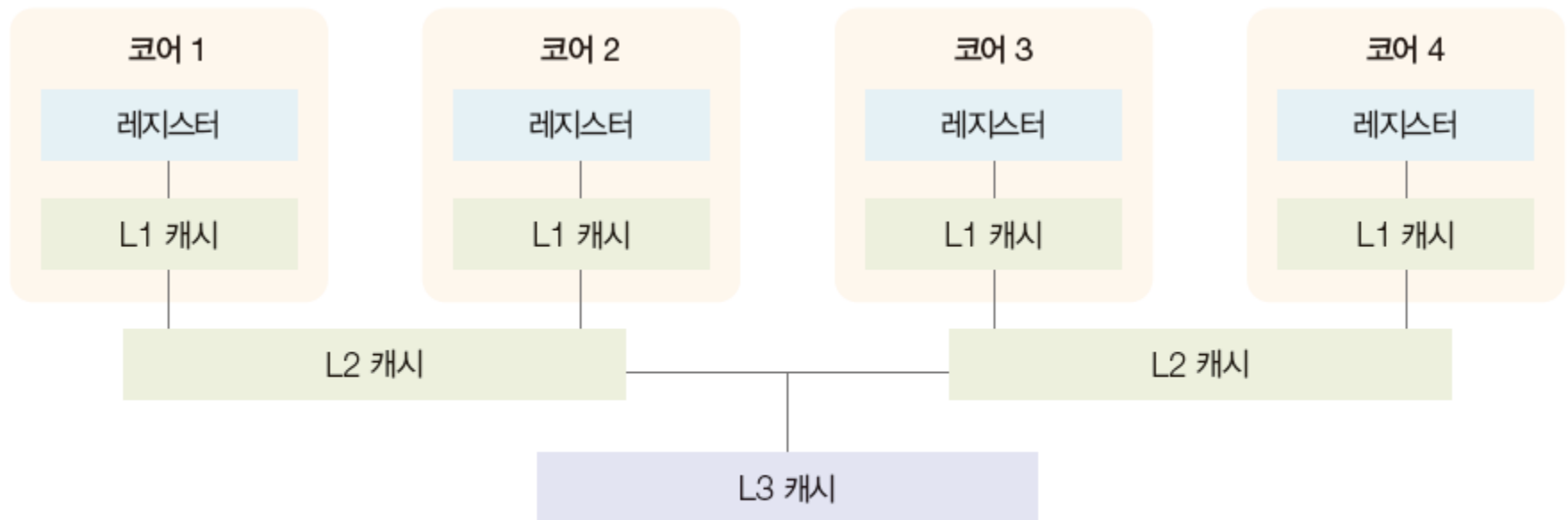


그림 2-28 캐시의 계층구조

03 컴퓨터 성능 향상 기술

3. 멀티프로세싱

◆ 명령어 병렬처리

- 한 개의 코어에서 여러 개의 명령어를 동시에 실행

03 컴퓨터 성능 향상 기술

3. 멀티프로세싱

◆ 멀티프로세싱 사례

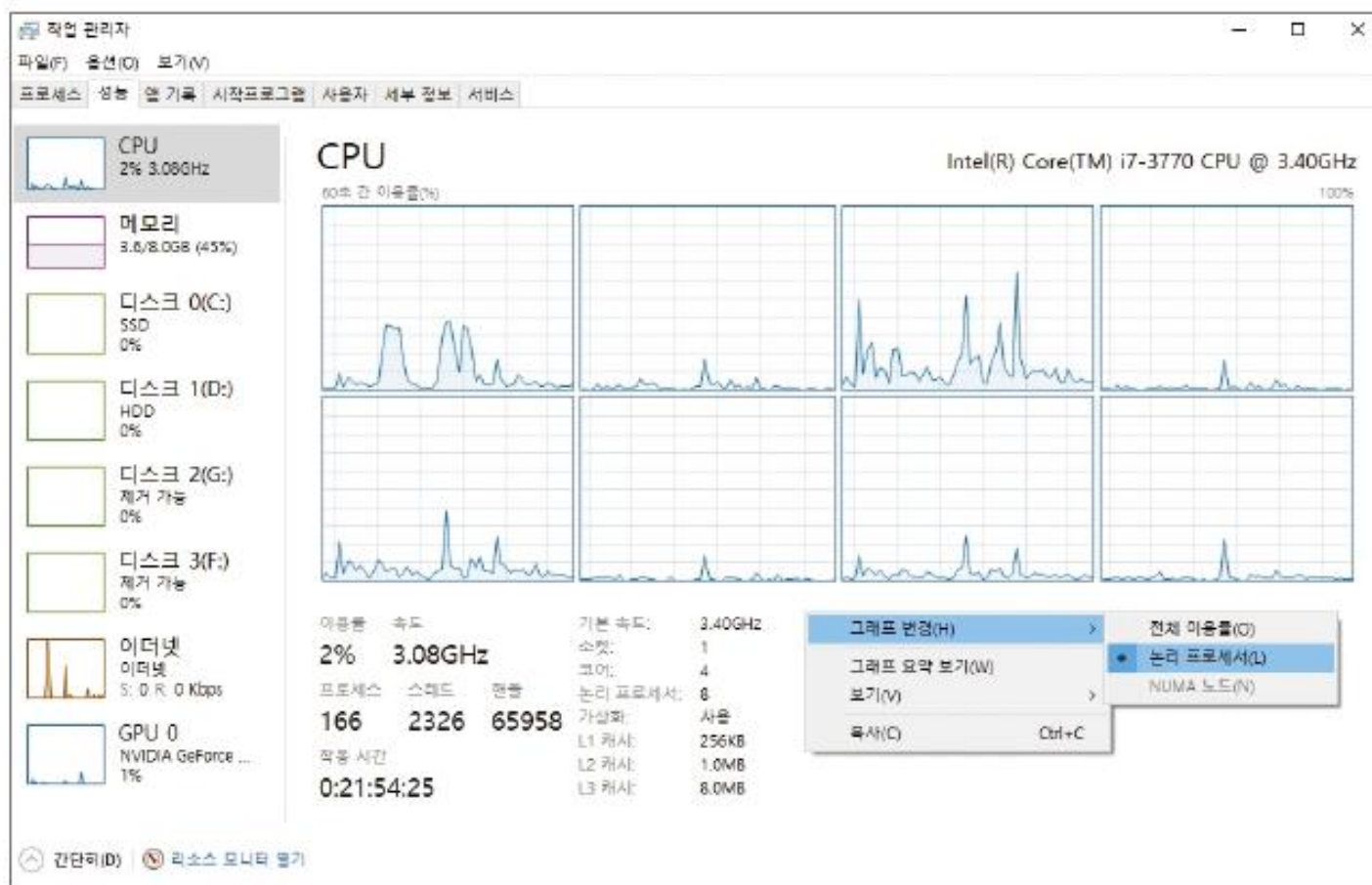


그림 2-29 멀티코어와 명령어 병렬처리가 적용된 CPU

03 컴퓨터 성능 향상 기술

4. [심화] 버퍼의 문제와 해결 방안

◆ 버퍼 오버플로와 scanf()

- C 언어에서 scanf() 함수로 사용자 입력을 받을 때 버퍼 오버플로 때문에 경고를 띄움
- 버퍼에 할당된 메모리보다 많이 입력했을 때
다른 메모리 영역에 초과 데이터를 덮어써버릴 수 있음

이름	코드	설명
	C4996	'scanf': This function or variable may be unsafe. Consider using scanf_s instead. To disable deprecation, use _CRT_SECURE_NO_WARNINGS. See online help for details.

그림 2-30 비주얼 스튜디오의 scanf() 사용 경고 메시지

03 컴퓨터 성능 향상 기술

코드 2-4 키보드 버퍼 때문에 scanf() 동작에 문제 발생

```
01  #include <stdio.h>
02
03  int main() {
04      int num1;
05      char c1;
06
07      printf("숫자 입력: ");
08      scanf("%d", &num1);
09      printf("숫자:[%d]\n", num1);
10
11      printf("문자 입력: ");
12      scanf("%c", &c1);
13      printf("문자:[%c]\n", c1);
14
15      return 0;
16  }
```

03 컴퓨터 성능 향상 기술

4. [심화] 버퍼의 문제와 해결 방안

◆ 키보드 버퍼와 scanf()

- 사용자로부터 연달아 입력받을 때, 첫 번째 입력 데이터를 가져간 후에 버퍼에 남아있는 데이터가 두 번째 입력 데이터와 함께 출력됨



```
숫자 입력: 100  
숫자:[100]  
문자 입력: 문자:[  
]
```

03 컴퓨터 성능 향상 기술

4. [심화] 버퍼의 문제와 해결 방안

◆ 모니터 버퍼

- 유닉스에서 모니터에 출력할 문자열은 마지막에 반드시 `\n`을 포함

◆ 하드웨어 안전 제거와 플러시

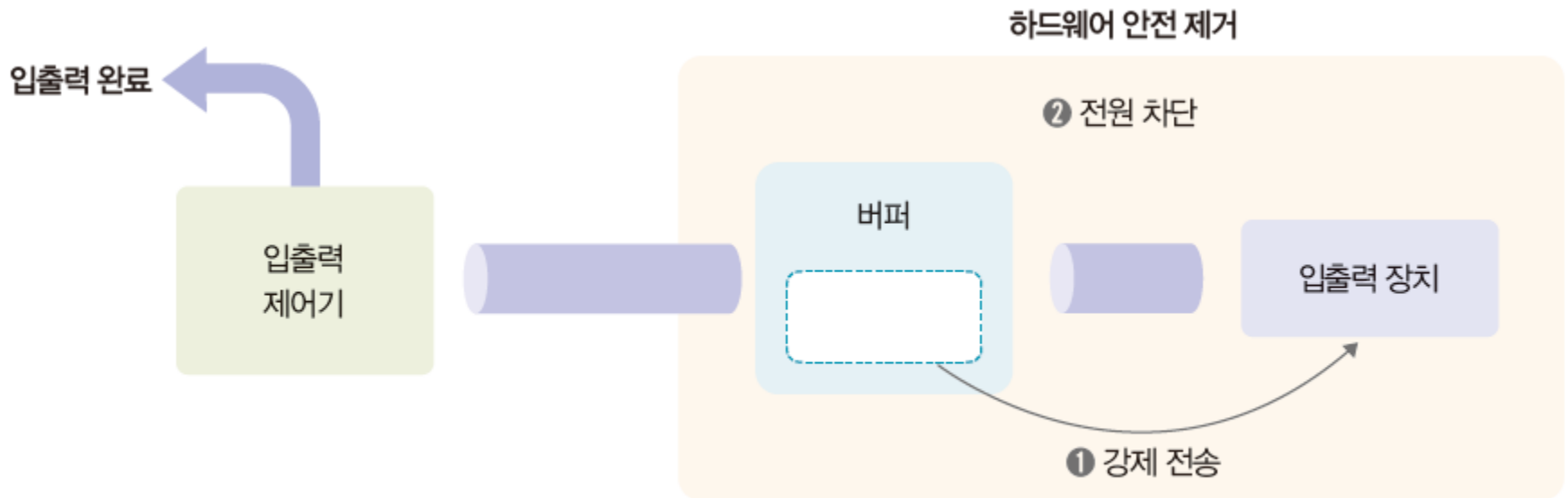


그림 2-31 하드웨어 안전 제거

03 컴퓨터 성능 향상 기술

4. [심화] 버퍼의 문제와 해결 방안

◆ 키보드 버퍼와 보안

- 해커가 키보드 버퍼에 저장된 데이터를 훔칠 수 있음
- 키보드 보안 프로그램, 가상 키보드



그림 2-32 가상 키보드

Thank You!